Participation Activity 8

Solution

Last Name: _____

First Name: _____

Std #:

- Consider our ISA x295++ from our A7 with the following components and modifications:
  - Memory model: 2^m x n = 2^{16} × 16 bits **->** This means that memory address is 16 bits in length and that Memory Address resolution is 16 bits, which is confirmed next
  - Address resolution: 16 bits **->** This means that every 16-bit memory chunck has a unique memory address, i.e., this "machine" is not a byte-addressable "machine" but a word-addressable "machine". How do I know that a word is 16 bits in length and that this is a word-addressable machine? See next line!
  - 8 × 16-bit registers -> (`r0` ↔ `r7`) **->** This means that a word is 16 bits in length, because word length is determined by the length of a register. In this ISA, a register can hold a data value or a memory address. How do I know this? Looking at the instruction **`LOAD offset(rA), rC`** we see that **`rA`** holds the memory address of the first element of array A and that **`rC`** will hold a data value once the instruction is executed. And since these two registers are 16 bits in length, this means that memory addresses as well as data are all 16 bits in length. See Data type below
  - Memore access mode: Base + Displacement mode -> i.e., base register + offset
  - Data type: `int` **->** This is the only data type in this ISA. This means that the only data type that a register can hold is an `int` and we already know (from the item above) that a register is 16 bits in length, therefore an `int` is 16 bits in length.
  - The assembly instruction LOAD in this IS has this syntax: **`LOAD offset(rA), rC`** and this meaning: `rC <- M[rA+offset]`
  - I can use the above LOAD instruction in my assembly code to step through an array A of `int` elements (i.e., 16 bit element) and load each of A's elements, where the source register **`rA`** contains the memory address of the first element of array A.
  - Imagine the array A contains 12 `int` elements. If I wanted to load A's **fifth** element into the destination register **`rC`**, to which value would I set **`offset`** in the above LOAD instruction used in my assembly code?
  - Answer: 4 **->** A, the name of the array, contains the memory address of its first element. The elements of A are of data type `int` and this data type is 16 bits in length. Since the memory address resolution is 16 bits (every chunck of 16 bits gets a unique memory address) and that, to get from the first element of array A to its fifth element, we need to "jump" over 4 elements (each with a unique memory address which increases by 1 for each array element), therefore, in order to "jump over" 4 elements, we need to add an offset fo 4 to the memory address contained in A.